

Playwright — Chapitre 3

Manipuler les composants UI avec Playwright

Thèmes du chapitre

1. Listes déroulantes statiques
2. Boutons radio
3. Cases à cocher
4. Assertions avec `expect`
5. `async / await` dans les assertions
6. Validation des attributs HTML
7. `textContent()` VS `inputValue()`
8. Fenêtres et onglets enfants
9. `Promise.all()`
10. `context.waitForEvent("page")`

1. Objectifs du chapitre

À la fin de ce chapitre, vous serez capable de :

- Sélectionner une option dans un élément `<select>`.
- Manipuler des boutons radio.
- Manipuler des cases à cocher.
- Vérifier l'état d'un élément.
- Utiliser `isChecked()`.
- Utiliser `toBeChecked()`.
- Comprendre `toBeTruthy()` et `toBeFalsy()`.
- Vérifier un attribut HTML avec `toHaveAttribute()`.
- Comprendre pourquoi `await` apparaît dans certaines assertions.
- Récupérer la valeur d'un champ avec `inputValue()`.
- Différencier `textContent()` et `inputValue()`.
- Gérer une nouvelle fenêtre ou un nouvel onglet.
- Attendre l'ouverture d'une nouvelle `Page`.
- Utiliser `Promise.all()` pour synchroniser plusieurs opérations.
- Réutiliser une information récupérée dans une nouvelle page sur la page d'origine.

2. Éléments HTML manipulés

Ce chapitre utilise principalement quatre types de composants d'interface.

Liste déroulante

```
xml
<select>
  <option value="student">Student</option>
  <option value="consult">Consult</option>
</select>
```

Bouton radio

```
xml
<input type="radio">
```

Case à cocher

```
xml
<input type="checkbox">
```

Champ de saisie

```
xml
<input id="username">
```

Playwright ne manipule pas uniquement des boutons : il peut interagir avec de nombreux éléments HTML et vérifier leur état ou leurs attributs.

3. Listes déroulantes statiques

3.1 Définition

En HTML, une liste déroulante statique est généralement un élément `<select>` contenant plusieurs éléments `<option>`.

```
html
<select class="form-control">
  <option value="student">Student</option>
  <option value="consult">Consult</option>
</select>
```

Playwright fournit une méthode dédiée à ce type de composant :

```
typescript
selectOption()
```

3.2 Sélectionner une option

```
typescript
const dropdown = page.locator("select.form-control");

await dropdown.selectOption("consult");
```

Le sélecteur `select.form-control` signifie :

Trouver un élément `<select>` possédant la classe CSS `form-control`.

La méthode suivante sélectionne l'option dont l'attribut `value` vaut `consult` :

```
typescript
await dropdown.selectOption("consult");
```

Elle cible donc :

```
xml
<option value="consult">Consult</option>
```

3.3 Exemple complet

```
typescript
import { test } from '@playwright/test';

test("Static Dropdown", async ({ page }) => {
  // Ouvrir l'application.
  await page.goto(
    "https://rahulshettyacademy.com/loginpagePractise/"
  );

  // Localiser la liste déroulante.
  const dropdown = page.locator("select.form-control");

  // Sélectionner l'option Consult.
  await dropdown.selectOption("consult");
});
```

`selectOption()` doit être utilisé avec de vrais éléments HTML `<select>`. Les dropdowns personnalisés peuvent nécessiter un clic puis la sélection manuelle d'une option.

4. Boutons radio

Un bouton radio permet généralement de sélectionner une seule option au sein d'un groupe.

```
xml
<input type="radio">
<input type="radio">
```

Dans l'exemple du cours, les boutons radio sont localisés avec :

```
typescript
page.locator(".radiotextsty")
```

Comme ce sélecteur peut correspondre à plusieurs éléments, Playwright propose notamment `last()` et `nth()`.

4.1 last() et nth()

```
typescript
locator.last()
```

Sélectionne le dernier élément correspondant au locator.

```
typescript
locator.nth(1)
```

Sélectionne l'élément situé à l'index 111.

Les index commencent à 000 :

- `nth(0)` : premier élément
- `nth(1)` : deuxième élément
- `nth(2)` : troisième élément

4.2 Cliquer sur un bouton radio

```
typescript
await page.locator(".radiotextsty").last().click();
```

Ce code suit le raisonnement suivant :

```
text
Locator → last() → click()
```

Autrement dit : Playwright trouve tous les éléments `.radiotextsty`, sélectionne le dernier, puis clique dessus.

4.3 Vérifier son état

```
typescript
await expect(
  page.locator(".radiotextsty").last()
).toBeChecked();
```

Cette assertion signifie :

Le dernier bouton radio doit être sélectionné.

Si ce n'est pas le cas, le test échoue.

5. isChecked() et toBeChecked()

Ces deux méthodes concernent l'état des boutons radio et des cases à cocher, mais elles n'ont pas le même objectif.

Méthode	Objectif	Résultat
isChecked()	Récupérer l'état actuel	true ou false
toBeChecked()	Vérifier qu'un élément est coché	Assertion Playwright

Lire l'état

```
typescript
const radio = page.locator(".radiotextsty").last();

console.log(await radio.isChecked());
```

Vérifier l'état

```
typescript
await expect(radio).toBeChecked();
```

isChecked() récupère une information, tandis que toBeChecked() valide une condition attendue.

6. Assertions avec expect

Un test automatisé ne doit pas seulement réaliser des actions. Il doit également vérifier le résultat de ces actions.

```
typescript
await radio.click();
```

Ce code demande uniquement de cliquer sur le bouton radio.

```
typescript
await expect(radio).toBeChecked();
```

Ce code vérifie que le bouton radio est réellement sélectionné après l'action.

C'est l'assertion qui transforme une simple automatisation en véritable test.

7. Cases à cocher

Une case à cocher peut être sélectionnée ou désélectionnée.

```
text
☐ Non sélectionnée
☒ Sélectionnée
```

Exemple :

```
typescript
await page.locator("#terms").click();

await expect(page.locator("#terms")).toBeChecked();
```

7.1 Cocher et décocher explicitement

Playwright propose des méthodes plus explicites que `click()` :

```
typescript
await page.locator("#terms").check();

await page.locator("#terms").uncheck();
```

Exemple complet :

```
typescript
const terms = page.locator("#terms");

await terms.check();
await expect(terms).toBeChecked();

await terms.uncheck();
expect(await terms.isChecked()).toBeFalse();
```

8. `toBeTruthy()` et `toBeFalsy()`

Ces assertions sont utilisées avec des valeurs booléennes ou des valeurs interprétées comme vraies ou fausses en JavaScript.

```
typescript
expect(true).toBeTruthy();

expect(false).toBeFalsy();
```

Dans le cas d'une checkbox :

```
typescript
expect(await checkbox.isChecked()).toBeFalsy();
```

Cela signifie :

Récupérer l'état de la case à cocher et vérifier qu'il vaut `false`.

9. Comprendre `await` dans les assertions

Deux syntaxes proches correspondent à deux mécanismes différents.

9.1 Assertion Playwright sur un locator

```
typescript
await expect(locator).toBeChecked();
```

Ici, Playwright attend automatiquement que la condition soit satisfaite, jusqu'à expiration du délai prévu.

Le `await` attend donc la fin de l'assertion.

9.2 Assertion JavaScript sur une valeur récupérée

```
typescript
expect(await locator.isChecked()).toBeFalsy();
```

Ici, `locator.isChecked()` est une opération asynchrone. Il faut donc attendre sa réponse avant de transmettre le booléen à `expect`.

Le déroulement est le suivant :

```
text
await locator.isChecked() → false
expect(false).toBeFalsy()
```

Les deux modèles à retenir

```
typescript
// Assertion Playwright sur un élément.
await expect(locator).toBeChecked();
typescript
// Assertion JavaScript sur une valeur récupérée.
expect(await locator.isChecked()).toBeFalsy();
```

10. Vérifier un attribut HTML

Playwright permet de valider les attributs d'un élément grâce à `toHaveAttribute()`.

```
typescript
```

```
const documentLink = page.locator(
  "[href*='documents-request']"
);

await expect(documentLink)
  .toHaveAttribute("class", "blinkingText");
```

Cette assertion vérifie que l'élément possède l'attribut suivant :

```
xml
class="blinkingText"
```

10.1 Le sélecteur [href*='documents-request']

```
typescript
[href*='documents-request']
```

Le symbole *= signifie « contient ».

Ce sélecteur peut donc correspondre à :

```
xml
<a href="documents-request">
```

ou à :

```
xml
<a href="/something/documents-request">
```

11. textContent() et inputValue()

11.1 textContent()

`textContent()` récupère le contenu textuel présent entre les balises d'un élément.

```
xml
<div>Hello World</div>
typescript
const text = await page.locator("div").textContent();
```

Résultat :

```
text
Hello World
```

11.2 Pourquoi ne pas utiliser textContent() sur un <input> ?

Considérons l'élément suivant :


```
xml
<input id="username" value="Alex">
```

Le texte `Alex` n'est pas placé entre une balise ouvrante et une balise fermante. Il se trouve dans l'attribut `value`.

Pour récupérer cette valeur, utilisez `inputValue()` :

```
typescript
const value = await page.locator("#username").inputValue();
```

Résultat :

```
text
Alex
```

Méthode

Utilisation

`textContent()` Texte contenu dans un élément

`inputValue()` Valeur d'un `<input>`, `<textarea>` ou champ similaire

12. Fenêtres et onglets enfants

Une application peut ouvrir une nouvelle fenêtre ou un nouvel onglet. Pour Playwright, cette nouvelle interface est représentée par une nouvelle instance de `Page`.

```
text
Page principale
↓
Nouvelle fenêtre ou nouvel onglet
↓
Nouvelle Page Playwright
```

12.1 Browser, Context et Page

```
typescript
const context = await browser.newContext();

const page = await context.newPage();
```

La structure est la suivante :

```
text
Browser
↓
Context
↓
Page principale
```

Le `Browser Context` regroupe les pages liées à une même session de navigation.

13. Attendre une nouvelle page

Supposons qu'un lien ouvre une nouvelle page :

```
typescript
const documentLink = page.locator(
  "[href*='documents-request']"
);
```

Le clic déclenche l'ouverture :

```
typescript
await documentLink.click();
```

Pour attendre l'apparition de la nouvelle page, utilisez :

```
typescript
context.waitForEvent("page")
```

Cette instruction signifie :

Attendre la création d'une nouvelle Page dans ce Browser Context.

14. Utiliser Promise.all()

La méthode recommandée consiste à préparer l'écoute de l'événement avant de déclencher le clic.

```
typescript
const [newPage] = await Promise.all([
  context.waitForEvent("page"),
  documentLink.click()
]);
```

Les deux opérations sont lancées ensemble :

- Playwright attend l'ouverture d'une page.
- Le clic sur le lien déclenche cette ouverture.

14.1 Pourquoi éviter cette approche ?

```
typescript
await documentLink.click();
```

```
const newPage = await context.waitForEvent("page");
```

Cette écriture peut produire un problème de synchronisation : la page peut être créée avant que Playwright ne commence à attendre l'événement.

14.2 Pourquoi Promise.all() est fiable

```
typescript
const [newPage] = await Promise.all([
  context.waitForEvent("page"),
  documentLink.click()
]);
```

Cette approche installe l'écouteur immédiatement et lance le clic dans le même temps.

14.3 Comprendre le destructuring

`Promise.all()` retourne un tableau de résultats.

Conceptuellement :

```
typescript
[
  nouvellePage,
  resultatDuClick
]
```

L'écriture suivante récupère directement le premier résultat :

```
typescript
const [newPage] = await Promise.all([...]);
```

`newPage` contient donc la nouvelle page ouverte.

15. Exploiter une nouvelle page

Une fois récupérée, `newPage` s'utilise comme n'importe quel objet `page`.

```
typescript
const text = await newPage
  .locator(".red")
  .textContent();
```

Par exemple, si le texte récupéré est :

```
text
Please email us at support@example.com for help
```

Vous pouvez extraire le domaine :

```
typescript
const domain = text.split("@")[1].split(" ")[0];
```

Résultat :

```
text
example.com
```

Vous pouvez ensuite revenir sur la page d'origine et injecter cette valeur dans un champ :

```
typescript
const username = page.locator("#username");

await username.fill(domain);
```

Le scénario global est donc :

```
text
Page principale
↓
Clic sur un lien
↓
Nouvelle page
↓
Récupération d'une information
↓
Retour sur la page principale
↓
Réutilisation de l'information
```

16. Exemple complet : composants UI

```
typescript
import { expect, test } from '@playwright/test';

test("UI Controls", async ({ page }) => {
  // Ouvrir la page de connexion.
  await page.goto(
    "https://rahulshettyacademy.com/loginpagePractise/"
  );

  // Localiser la liste déroulante.
  const dropdown = page.locator("select.form-control");

  // Sélectionner l'option Consult.
  await dropdown.selectOption("consult");

  // Localiser le dernier bouton radio.
  const radioButton = page.locator(".radiotextsty").last();

  // Sélectionner le bouton radio.
  await radioButton.click();

  // Accepter la fenêtre pop-up.
  await page.locator("#okayBtn").click();

  // Lire l'état du bouton radio.
  console.log(await radioButton.isChecked());

  // Vérifier qu'il est sélectionné.
  await expect(radioButton).toBeChecked();
});
```

```

// Localiser la checkbox des conditions.
const terms = page.locator("#terms");

// Cocher la checkbox.
await terms.check();

// Vérifier qu'elle est cochée.
await expect(terms).toBeChecked();

// Décocher la checkbox.
await terms.uncheck();

// Vérifier qu'elle est décochée.
expect(await terms.isChecked()).toBeFalse();

// Localiser le lien vers les documents.
const documentLink = page.locator(
  "[href*='documents-request']"
);

// Vérifier l'attribut class du lien.
await expect(documentLink)
  .toHaveAttribute("class", "blinkingText");
});

```

17. Exemple complet : nouvelle fenêtre / nouvel onglet

```

typescript
import { test } from '@playwright/test';

test("Child Windows Handling", async ({ browser }) => {
  // Créer un contexte de navigation.
  const context = await browser.newContext();

  // Créer la page principale.
  const page = await context.newPage();

  // Ouvrir l'application.
  await page.goto(
    "https://rahulshettyacademy.com/loginpagePractise/"
  );

  // Localiser le lien qui ouvre une nouvelle page.
  const documentLink = page.locator(
    "[href*='documents-request']"
  );

  // Attendre l'ouverture de la nouvelle page pendant le clic.
  const [newPage] = await Promise.all([
    context.waitForEvent("page"),
    documentLink.click()
  ]);

  // Récupérer le texte présent dans la nouvelle page.

```

```

const text = await newPage
  .locator(".red")
  .textContent();

// Extraire le domaine de l'adresse e-mail.
const domain = text
  .split("@")[1]
  .split(" ")[0];

// Afficher le domaine.
console.log(domain);

// Revenir sur la page principale.
const username = page.locator("#username");

// Saisir le domaine dans le champ username.
await username.fill(domain);

// Vérifier la valeur saisie.
console.log(await username.inputValue());
});

```

18. Concepts essentiels à retenir

Concept	Exemple	Objectif
Liste déroulante	<code>await dropdown.selectOption("consult")</code>	Sélectionner une option dans un <code><select></code>
Bouton radio	<code>await radio.click()</code>	Sélectionner une option
Assertion de sélection	<code>await expect(radio).toBeChecked()</code>	Vérifier l'état sélectionné
Checkbox	<code>await checkbox.check()</code>	Cocher explicitement
Décocher	<code>await checkbox.uncheck()</code>	Décocher explicitement
Lire un état	<code>await checkbox.isChecked()</code>	Retourner <code>true</code> ou <code>false</code>
Attribut HTML	<code>toHaveAttribute("class", "blinkingText")</code>	Vérifier un attribut
Texte	<code>await locator.textContent()</code>	Lire le contenu textuel
Champ de saisie	<code>await username.inputValue()</code>	Lire la valeur d'un input
Nouvelle page	<code>context.waitForEvent("page")</code>	Attendre un nouvel onglet ou une fenêtre
Synchronisation	<code>await Promise.all([...])</code>	Attendre un événement et l'action qui le déclenche

19. Schéma mental

PLAYWRIGHT



Composants UI



Dropdown	Radio button	Checkbox	
selectOption	click	check	
	isChecked	uncheck	
	toBeChecked	isChecked	

Multi-pages



Browser Context



waitForEvent("page")



Promise.all()



newPage

20. Checklist du chapitre

- Je sais ce qu'est un élément `<select>`.
- Je sais utiliser `selectOption()`.
- Je sais utiliser `last()`.
- Je sais utiliser `nth()`.
- Je sais cliquer sur un bouton radio.
- Je sais utiliser `isChecked()`.
- Je sais utiliser `toBeChecked()`.
- Je sais cocher une checkbox avec `check()`.
- Je sais décocher une checkbox avec `uncheck()`.
- Je comprends `toBeTruthy()`.
- Je comprends `toBeFalsy()`.
- Je comprends l'utilisation de `await` dans les opérations asynchrones.
- Je sais utiliser `toHaveAttribute()`.
- Je comprends le sélecteur `[attribute*='value']`.
- Je sais différencier `textContent()` et `inputValue()`.
- Je sais pourquoi `inputValue()` est utilisé pour un `<input>`.
- Je comprends ce qu'est une Child Window.
- Je comprends qu'un nouvel onglet est une nouvelle Page.
- Je sais utiliser `context.waitForEvent("page")`.
- Je comprends pourquoi l'événement doit être écouté avant le clic.
- Je comprends l'intérêt de `Promise.all()`.
- Je comprends le destructuring `const [newPage]`.
- Je sais récupérer une donnée depuis une deuxième page.
- Je sais réutiliser cette donnée sur la première page.

21. Les cinq notions prioritaires

1. Chaque composant HTML a ses méthodes Playwright adaptées

```
typescript
selectOption()
check()
uncheck()
click()
```

2. Il faut distinguer lecture d'une valeur et assertion

```
typescript
await locator.isChecked();
typescript
await expect(locator).toBeChecked();
```

3. `inputValue()` est la bonne méthode pour la valeur d'un champ

```
typescript
await page.locator("#username").inputValue();
```

4. Une nouvelle fenêtre ou un nouvel onglet est une nouvelle `Page`

```
typescript
context.waitForEvent("page")
```

5. `Promise.all()` synchronise l'écoute d'un événement avec le clic qui le déclenche

```
typescript
const [newPage] = await Promise.all([
  context.waitForEvent("page"),
  documentLink.click()
]);
```